TF4015 – Sistem Kontrol Diskrit

Praktikum

Modul 3

Kontrol Posisi Motor DC

Tim Pengajar TF4015

Program Studi Teknik Fisika
Fakultas Teknologi Industri
Institut Teknologi Bandung

Semester I 2026/2027

Daftar Isi

1 Pendahuluan **1**
1.1 Latar Belakang . . . 1
1.2 Tujuan Praktikum . . . 1
1.3 Peralatan dan Perangkat Lunak . . . 1
1.4 Persiapan Praktikum . . . 2

2 Teori Dasar **2**
2.1 Hubungan Kecepatan dan Posisi . . . 2
2.2 Model Sistem Posisi . . . 2
2.3 Batas Error Posisi . . . 2
2.4 Penalaan Ziegler–Nichols Kalang Tertutup . . . 3
2.5 Ketidaklinieran pada Kontrol Posisi . . . 3

3 Keselamatan Kerja **4**

4 Prosedur Praktikum **4**
4.1 Percobaan 1: Pemodelan Sistem dan Simulasi . . . 4
4.2 Percobaan 2: Validasi Faktor Konversi Kecepatan–Posisi . . . 5
4.3 Percobaan 3: Persiapan Sistem Kontrol . . . 6
4.4 Percobaan 4: Implementasi Kontrol Posisi . . . 6

5 Laporan dan Analisis **7**
5.1 Ketentuan Laporan . . . 7
5.2 Analisis . . . 8

6 Tugas Tambahan **9**

A Kode Program Pendukung **10**

1 Pendahuluan

1.1 Latar Belakang

Pengendalian posisi tidak kalah umum dibandingkan pengendalian kecepatan. Aplikasinya antara lain pengendalian bukaan katup, pintu atau palang otomatis, printer dan *3D printer*, serta mesin CNC. Pada Modul 2, kecepatan putar motor DC telah dikontrol menggunakan PID dengan penalaan Ziegler–Nichols, Cohen–Coon, dan penalaan halus. Pada Modul 3, besaran yang dikontrol adalah **posisi sudut** yang diukur oleh potensiometer.

Model posisi diturunkan dari model kecepatan Modul 1 dengan memperhatikan hubungan antara tachometer dan potensiometer pada kit. Berbeda dengan sistem kecepatan, sistem posisi memiliki sifat **integrator**: masukan konstan menghasilkan posisi yang terus bertambah. Sifat ini, bersama *deadzone*, waktu tunda, dan bacaan posisi yang berpindah dari 100% ke 0%, membuat kontrol posisi pada kit cukup menantang. Parameter awal pengontrol ditentukan dengan metode Ziegler–Nichols kalang tertutup melalui simulasi, lalu diterapkan, ditala, dan diuji terhadap gangguan.

1.2 Tujuan Praktikum

Setelah menyelesaikan Modul 3, praktikan diharapkan mampu:

1. Menurunkan model sistem posisi dari model sistem kecepatan dan memvalidasi faktor konversinya secara eksperimen.
2. Menala pengontrol posisi dengan metode Ziegler–Nichols kalang tertutup melalui simulasi.
3. Menyiapkan sistem kontrol posisi dengan memperhatikan ketidaklinieran kit, yaitu batas error, batas aksi kontrol, dan *deadzone*.
4. Mengimplementasikan kontrol posisi dan menguji performanya terhadap perubahan set point dan gangguan.

1.3 Peralatan dan Perangkat Lunak

Tabel 1: Daftar peralatan praktikum Modul 3

No	Nama Peralatan	Keterangan
1	Kit praktikum SKD (kit yang sama dengan modul sebelumnya)	Disediakan asisten
2	Laptop dengan Arduino IDE dan *library* Adafruit MCP4725	Milik praktikan
3	MATLAB (desktop atau MATLAB Online)	Milik praktikan
4	Perangkat lunak *spreadsheet*	Milik praktikan
5	Program: `5_control.ino` (versi Modul 2), `10_main_kontrol_posisi`	Unduh dari repositori
6	Skrip MATLAB `simulasi_kontrol_posisi.m`	Unduh dari repositori
7	Model final Modul 1 dan pengontrol terbaik Modul 2	Hasil praktikum sebelumnya

1.4 Persiapan Praktikum

1. Siapkan model final Modul 1 dan tabel spesifikasi kit (terutama masukan minimum putar motor).
2. Hitung faktor konversi C secara analitik (Bagian 2.1) dan jalankan skrip `simulasi_kontrol_posisi.m` dengan model kelompok.
3. Pahami program `10_main_kontrol_posisi` (Kode 1), terutama fungsi `wrap_error()`.
4. Baca prosedur dan bagian analisis, lalu bagi tugas dalam kelompok. Percobaan 1 dapat dikerjakan sebelum praktikum.

2 Teori Dasar

2.1 Hubungan Kecepatan dan Posisi

Bacaan potensiometer dinyatakan dalam persen satu putaran potensiometer, sedangkan bacaan tachometer dinyatakan dalam persen skala ADC. Misalkan kecepatan terukur v (%) berbanding lurus dengan kecepatan sudut motor ω_m (rpm), dengan kecepatan maksimum terukur v_{maks} setara dengan ω_{maks}. Dengan perbandingan roda gigi motor terhadap potensiometer 16 : 100 dan tanpa slip, laju perubahan posisi potensiometer p (%) adalah

$$\frac{dp}{dt} = C\,v, \qquad C = \frac{\omega_{maks}}{v_{maks}} \cdot \frac{1}{60\,000} \cdot \frac{16}{100} \cdot 100 \quad [\%/\text{ms per } \%], \tag{1}$$

dengan faktor $1/60\,000$ mengubah rpm menjadi putaran per milidetik. Bila ω_{maks} tidak diketahui, asumsikan kecepatan maksimum hasil Modul 1 setara dengan 6400 rpm. Karena asumsi ini belum tentu tepat, nilai C juga akan diukur langsung pada Percobaan 2.

2.2 Model Sistem Posisi

Dengan model kecepatan FOPDT dari Modul 1 dan Persamaan (1), model posisi adalah

$$G_p(s) = \frac{P(s)}{U(s)} = \frac{CK\,e^{-\theta s}}{s\,(\tau s + 1)}. \tag{2}$$

Faktor $1/s$ menunjukkan bahwa sistem posisi adalah **integrator**: masukan step menghasilkan posisi yang naik seperti *ramp*, dan sistem kalang terbuka tidak memiliki keadaan tunak. Sistem bertipe satu ini, bila dikontrol dengan pengontrol P, secara teoretis tidak memiliki offset terhadap set point step. Dalam praktik, *deadzone* dan gesekan dapat menyebabkan error tunak kecil.

2.3 Batas Error Posisi

Potensiometer dapat berputar tanpa batas, sehingga bacaan posisi berpindah dari sekitar 100% ke 0% (dan sebaliknya). Akibatnya, selisih set point dan posisi dapat bernilai hingga ±100%, padahal jarak terpendek ke set point tidak pernah lebih dari

setengah putaran. Error posisi perlu dibatasi pada rentang −50 hingga 50%:

$$e = \text{wrap}(r - p) = \begin{cases} r - p - 100, & r - p > 50, \\ r - p + 100, & r - p < -50, \\ r - p, & \text{lainnya.} \end{cases} \tag{3}$$

Dengan demikian motor selalu bergerak melalui jalur terpendek menuju set point.

2.4 Penalaan Ziegler–Nichols Kalang Tertutup

Metode Ziegler–Nichols kurva reaksi pada Modul 2 tidak dapat digunakan langsung karena respons kalang terbuka sistem posisi tidak mencapai keadaan tunak. Sebagai gantinya digunakan metode **kalang tertutup** (metode gain kritis):

1. Gunakan pengontrol P saja, lalu naikkan K_p hingga respons berosilasi dengan amplitudo tetap. Nilai ini adalah gain kritis K_{cr}.
2. Ukur perioda osilasi tersebut, yaitu perioda kritis P_{cr}.
3. Hitung parameter pengontrol dengan Tabel 2.

Pada kit praktikum, mencari K_{cr} secara langsung berisiko merusak kit, sehingga langkah ini dilakukan pada simulasi.

Tabel 2: Parameter Ziegler–Nichols kalang tertutup

Kontrol	K_p	T_i	T_d
P	$0{,}5\,K_{cr}$	–	–
PI	$0{,}45\,K_{cr}$	$P_{cr}/1{,}2$	–
PID	$0{,}6\,K_{cr}$	$P_{cr}/2$	$P_{cr}/8$
PID *overshoot* kecil	$0{,}33\,K_{cr}$	$P_{cr}/2$	$P_{cr}/3$
PID tanpa *overshoot*	$0{,}2\,K_{cr}$	$P_{cr}/2$	$P_{cr}/3$

2.5 Ketidaklinieran pada Kontrol Posisi

Beberapa ketidaklinieran berikut perlu ditangani agar kontrol posisi berhasil:

- **Batas aksi kontrol**. Pada masukan besar, motor berputar terlalu cepat sehingga posisi sulit diamati dan dikontrol. Aksi kontrol perlu dibatasi pada rentang $\pm U_{maks}$ yang ditentukan dari pengamatan kalang terbuka (Percobaan 3).
- **Kompensasi *deadzone***. Untuk kontrol posisi, yang penting adalah motor mulai bergerak, bukan tachometer mulai membaca. Nilai *deadzone* yang sesuai dapat berbeda dengan yang digunakan pada Modul 2.
- **Osilasi kecil di sekitar set point**. Dengan kompensasi *deadzone*, error yang sangat kecil tetap menghasilkan masukan sebesar *deadzone* sehingga motor dapat bergerak bolak-balik di sekitar set point (*limit cycle*). Salah satu solusinya adalah pita error (`ErrorBand`): aksi kontrol dibuat nol bila $|e|$ cukup kecil.
- **Bagian integral**. Karena sistem sudah memiliki integrator, aksi integral sering tidak diperlukan dan justru dapat menambah *overshoot*.

3 Keselamatan Kerja

Seluruh ketentuan keselamatan kerja dan SOP pada modul sebelumnya tetap berlaku. Perhatikan pula hal-hal berikut:

1. Sistem posisi sangat sensitif. Mulailah dengan K_p kecil dan batas aksi kontrol kecil, lalu naikkan bertahap.
2. Bila motor berosilasi keras atau berputar terus-menerus, segera cabut kabel listrik 220 V.
3. Pada uji gangguan, putar roda gigi potensiometer perlahan dan sebentar saja dengan memegang tepi piringan. Jauhkan jari dari titik pertemuan kedua roda gigi.
4. Jangan mengubah trimpot penguatan OpAmp.

4 Prosedur Praktikum

Prosedur Modul 3 terdiri atas empat percobaan. Percobaan 1 dikerjakan di MATLAB dan dapat dikerjakan sebelum praktikum atau secara paralel. Simpan seluruh data mentah ke file teks.

4.1 Percobaan 1: Pemodelan Sistem dan Simulasi

1. Hitung faktor konversi C dengan Persamaan (1), lalu lengkapi kolom “analitik” pada Tabel 3. Gunakan waktu cacah pertama dari Modul 1.
2. Isi parameter model pada skrip `simulasi_kontrol_posisi.m` (Kode 2) dan amati respons kalang terbuka posisi.
3. Cari K_{cr} dan P_{cr} secara manual dengan menaikkan `KpUji` pada bagian 3 skrip hingga respons berosilasi dengan amplitudo tetap. Bandingkan dengan nilai yang dihitung skrip dari model diskrit.
4. Lengkapi Tabel 4 dengan parameter Ziegler–Nichols kalang tertutup dan Tabel 5 dengan performa simulasinya.

Tabel 3: Model sistem kecepatan dan posisi kit praktikum

No	Parameter	Analitik	Eksperimen	Satuan
1	Waktu cacah T_s			ms
2	Penguatan K (Modul 1)			–
3	Konstanta waktu τ (Modul 1)			ms
4	Waktu tunda θ (Modul 1)			ms
5	Faktor konversi C			%/(ms·%)
6	Model posisi $G_p(s)$			–

Tabel 4: Parameter kontrol Ziegler–Nichols kalang tertutup

Kontrol	K_p	T_i **(ms)**	T_d **(ms)**	K_i **(1/ms)**	K_d **(ms)**
P		–	–	–	–
PI			–		–
PID					
PID *overshoot* kecil					
PID tanpa *overshoot*					
K_{cr} =	P_{cr} =		ms		

Tabel 5: Performa simulasi kontrol posisi

Kontrol	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**
P					
PI					
PID					
PID *overshoot* kecil					
PID tanpa *overshoot*					

4.2 Percobaan 2: Validasi Faktor Konversi Kecepatan–Posisi

1. Ganti tab `9_main.ino` dengan program `10_main_kontrol_posisi` dan atur `ClosedLoop = false`. Unggah, verifikasi tanpa daya motor, lalu hubungkan kabel listrik 220 V.
2. Kirim masukan `20` melalui *Serial Monitor*. Setelah kecepatan tunak, rekam data selama sekitar 2 detik, lalu kirim `0`.
3. Ulangi untuk masukan `30` dan `40`.
4. Untuk setiap masukan, buka (*unwrap*) bacaan posisi sehingga tidak berpindah dari 100 ke 0, lalu hitung kemiringan posisi terhadap waktu (%/ms) dengan regresi linier. Bagi kemiringan tersebut dengan rata-rata kecepatan terukur untuk memperoleh C.
5. Lengkapi Tabel 6, lalu isi kolom “eksperimen” pada Tabel 3 dengan rata-rata nilai C. Bila nilai eksperimen berbeda jauh dengan nilai analitik, gunakan nilai eksperimen untuk simulasi selanjutnya.

Tabel 6: Pengukuran faktor konversi kecepatan–posisi

Masukan (%)	**Kecepatan rata-rata (%)**	**Kemiringan posisi (%/ms)**	C **(%/(ms·%))**
20			
30			
40			
		Rata-rata	

4.3 Percobaan 3: Persiapan Sistem Kontrol

1. Tetap dengan `ClosedLoop = false`, kirim berbagai masukan pada rentang -100 hingga 100 (kembali ke `0` sebelum mengubah arah). Amati bacaan posisi menggunakan *Serial Plotter*.
2. Klasifikasikan pengamatan secara kualitatif, misalnya: motor tidak bergerak, posisi bergerak pelan dan mudah diamati, atau posisi bergerak terlalu cepat sehingga sulit diamati. Lengkapi Tabel 7.
3. Berdasarkan klasifikasi tersebut, tentukan batas aksi kontrol U_{maks} dan isikan pada `OutputMin` dan `OutputMax` di `setup()`.
4. Periksa kompensasi *deadzone*. Bila perlu, ubah `DeadZonePositif` dan `DeadZoneNegatif` sesuai masukan minimum putar motor (Modul 1) agar masukan kecil langsung menggerakkan posisi. Catat nilai yang digunakan.

Tabel 7: Klasifikasi pengamatan posisi terhadap masukan

No	**Rentang masukan (%)**	**Klasifikasi pengamatan**
1		
2		
3		
4		
5		

4.4 Percobaan 4: Implementasi Kontrol Posisi

1. Atur `ClosedLoop = true`, `TimeSampling` $= T_s$, dan `SetPointSequence = {50, 20, 50, 90, 20, 50}`. Program menahan setiap set point selama `HoldTime`.
2. Untuk setiap parameter pada Tabel 4, jalankan kontrol posisi dan simpan datanya. Mulailah dengan pengontrol yang paling kecil gainnya. Hitung performa untuk perubahan set point $50 \rightarrow 20$ dan $20 \rightarrow 50$, lalu lengkapi Tabel 8.
3. Lakukan penalaan halus pada pengontrol dengan performa terbaik. Pengontrol yang digunakan tidak harus P, PI, atau PID; PD atau modifikasi lain seperti pita error (`ErrorBand`) diperbolehkan. Catat setiap modifikasi beserta alasannya.

4. Apabila performa masih belum memuaskan setelah penalaan halus, modifikasi program lebih lanjut (misalnya kontrol *on-off* dengan histeresis, atau batas aksi kontrol yang berbeda untuk error besar dan kecil).
5. **Uji gangguan**: gunakan pengontrol terbaik. Setelah urutan set point selesai, set point terakhir (50) dipertahankan. Berikan gangguan dengan memutar roda gigi potensiometer sedikit secara manual, lalu lepaskan. Simpan respons pengontrolan terhadap gangguan.

Tabel 8: Performa implementasi kontrol posisi

Kontrol	**Step**	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**
P	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
PI	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
PID	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
PID OS kecil	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
PID tanpa OS	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
Penalaan halus	$50 \rightarrow 20$					
	$20 \rightarrow 50$					

5 Laporan dan Analisis

5.1 Ketentuan Laporan

Laporan dikumpulkan per kelompok melalui Edunex paling lambat dua minggu setelah praktikum. Nilai tambahan diberikan bila laporan dikumpulkan dalam satu minggu. Laporan berupa file `.pdf` yang dilampiri *spreadsheet* pengolahan data, data mentah (`.txt`), program Arduino, dan skrip MATLAB, digabung dalam satu file `.zip`. Laporan minimal memiliki susunan berikut: latar belakang, tujuan, alat, prosedur, analisis, kesimpulan, kontribusi, dan referensi.

Latar belakang, tujuan, alat, dan prosedur tidak boleh sama persis dengan modul ini. Analisis disusun dengan alur yang runtut dengan menjawab pertanyaan pada Bagian 5.2. Perhatikan penggunaan angka penting. Bagian kontribusi berisi Tabel 9.

Tabel 9: Kontribusi anggota kelompok

No	**NIM**	**Nama Lengkap**	**Kontribusi**
1			
2			
3			
4			
5			

5.2 Analisis

Pemodelan sistem dan simulasi.

1. Uraikan penurunan model posisi dari model kecepatan beserta asumsinya. Bandingkan faktor konversi C analitik dan eksperimen. Asumsi mana yang paling mungkin menyebabkan perbedaan?
2. Apa yang terjadi pada sistem posisi kalang terbuka ketika diberi masukan step? Mengapa demikian?
3. Mengapa metode Ziegler–Nichols kurva reaksi tidak dapat digunakan untuk sistem posisi?
4. Uraikan langkah-langkah memperoleh K_{cr} dan P_{cr} dalam simulasi. Apakah hasil pencarian manual sesuai dengan hasil yang dihitung dari model diskrit?
5. Berdasarkan simulasi, pengontrol mana yang memberikan performa terbaik? Mengapa aksi integral tidak banyak membantu pada sistem posisi?

Persiapan sistem kontrol.

6. Jelaskan klasifikasi pengamatan kalang terbuka dan bagaimana klasifikasi tersebut digunakan untuk menentukan batas aksi kontrol.
7. Apakah fungsi kompensasi *deadzone* perlu diubah untuk kontrol posisi? Jelaskan alasannya.
8. Jelaskan fungsi `wrap_error()` pada program kontrol posisi. Apa yang terjadi bila fungsi ini tidak digunakan ketika set point berubah dari 90 ke 20?

Implementasi kontrol.

9. Bagaimana hasil pengontrolan menggunakan parameter Ziegler–Nichols kalang tertutup? Bandingkan dengan hasil simulasi dan jelaskan penyebab perbedaannya.
10. Apakah kontrol posisi berhasil? Bila tidak, jelaskan penyebabnya. Bila berhasil, pengontrol mana yang menghasilkan performa terbaik?
11. Jelaskan penalaan halus dan modifikasi program yang dilakukan. Apakah performa menjadi lebih baik?
12. Apakah terjadi osilasi kecil di sekitar set point? Jelaskan penyebabnya dan bagaimana kelompok mengatasinya.
13. Tunjukkan performa terbaik yang diperoleh dan lengkapi Tabel 10.
14. Tunjukkan respons sistem dengan pengontrol terbaik terhadap gangguan manual. Apakah posisi kembali ke set point? Berapa lama waktu yang dibutuhkan?

Tabel 10: Performa terbaik implementasi kontrol posisi

K_p	K_i	K_d	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**

6 Tugas Tambahan

Tugas tambahan bersifat opsional dan merupakan persiapan Modul 4.

1. Tuliskan model sistem posisi dalam bentuk ruang keadaan kontinu dengan keadaan $x = [p \;\; v]^T$, lalu diskritkan dengan *zero-order hold*.
2. Kecepatan bertanda dapat diperoleh dari tachometer dan arah putar motor. Apa kelemahan cara ini? Usulkan cara lain untuk memperoleh kecepatan dari bacaan potensiometer.
3. Gunakan kecepatan terukur sebagai umpan balik tambahan pada kontrol posisi ($u = K_p e - K_v v$). Bandingkan hasilnya dengan pengontrol PD.

Daftar Pustaka

[1] K. Ogata, *Modern Control Engineering*, edisi ke-5. Upper Saddle River, NJ: Prentice Hall, 2010.

[2] D. E. Seborg, T. F. Edgar, D. A. Mellichamp, dan F. J. Doyle III, *Process Dynamics and Control*, edisi ke-4. Hoboken, NJ: Wiley, 2016.

A Kode Program Pendukung

Program utama berikut menggantikan isi tab `9_main.ino`. Tab `5_control.ino` yang digunakan adalah versi Modul 2.

Kode 1: Program utama `10_main_kontrol_posisi`

```
// ============================================================================
// Program utama Modul 3: kontrol posisi motor DC
// Ganti isi tab 9_main.ino dengan program ini.
//
// ClosedLoop = false : kalang terbuka, masukan motor dikirim melalui
//                      Serial Monitor (-100 hingga 100).
// ClosedLoop = true  : kontrol posisi dengan set point mengikuti
//                      SetPointSequence[]. Setelah urutan selesai, set point
//                      terakhir dipertahankan (untuk uji gangguan).
// Error posisi dibatasi pada rentang -50 hingga 50 karena bacaan posisi
// berpindah dari 100 ke 0 (dan sebaliknya).
// ============================================================================

unsigned long Time, StartTime, LastTime, StepStart;
unsigned int TimeSampling = 10, DeltaTime;   // waktu cacah (ms)
unsigned int HoldTime = 3000;                // lama tiap set point (ms)
bool ClosedLoop = true;
float ErrorBand = 0.0;                       // pita error (%) tempat keluaran
    dibuat nol

float SetPointSequence[] = {50, 20, 50, 90, 20, 50};
const byte SequenceLength = sizeof(SetPointSequence) / sizeof(SetPointSequence[0]);

byte Index = 0;
int InputSerial = 0;
float SetPoint = 0.0, Position = 0.0, Speed = 0.0, Error = 0.0;
float ManipulatedVariable = 0.0, Motor = 0.0;

float wrap_error(float e) {
  while (e > 50.0) e -= 100.0;
  while (e < -50.0) e += 100.0;
  return e;
}

void setup() {
  general_setup();
  sensor_setup();
  actuator_setup();

  // Parameter pengontrol (Ki dalam 1/ms, Kd dalam ms)
  Kp = 1.0;
  Ki = 0.0;
  Kd = 0.0;
  OutputMin = -40.0;   // batas aksi kontrol sesuai klasifikasi kalang terbuka
  OutputMax = 40.0;
  control_reset();

  Serial.println("Time\tDelta\tSP\tPos\tKec\tMV\tMotor");
  StartTime = millis();
  LastTime = StartTime;
  StepStart = StartTime;
}

void loop() {
```

```
  InputSerial = serial(InputSerial);
  Time = millis();
  if (Time - LastTime >= TimeSampling) {
    DeltaTime = Time - LastTime;
    LastTime = Time;

    Position = smooth_position_percentage();
    Speed = smooth_velocity_percentage();

    if (ClosedLoop) {
      SetPoint = SetPointSequence[Index];
      Error = wrap_error(SetPoint - Position);
      ManipulatedVariable = control(Error, TimeSampling);
      if (Error <= ErrorBand && Error >= -ErrorBand) ManipulatedVariable = 0.0;
    }
    else {
      SetPoint = 0.0;
      ManipulatedVariable = (float) InputSerial;
    }

    Motor = run_motor(compensation(ManipulatedVariable));

    Serial.print(Time - StartTime);
    Serial.print("\t");
    Serial.print(DeltaTime);
    Serial.print("\t");
    Serial.print(SetPoint);
    Serial.print("\t");
    Serial.print(Position);
    Serial.print("\t");
    Serial.print(Speed);
    Serial.print("\t");
    Serial.print(ManipulatedVariable);
    Serial.print("\t");
    Serial.println(Motor);

    if (ClosedLoop && Index < SequenceLength - 1 && Time - StepStart >= HoldTime) {
      StepStart = Time;
      Index++;
    }
  }
}
```

Kode 2: Skrip MATLAB `simulasi_kontrol_posisi.m`

```
%% simulasi_kontrol_posisi.m
% Simulasi kontrol posisi motor DC (Modul 3).
% Model posisi diturunkan dari model kecepatan (Modul 1):
%   G_p(s) = C * K * exp(-Theta s) / ( s (Tau s + 1) )
% dengan C adalah faktor konversi kecepatan (%) ke laju posisi (%/ms).
%
% Skrip ini:
%   1. mensimulasikan respons kalang terbuka posisi,
%   2. mensimulasikan kontrol P dengan Kp uji untuk mencari Kcr dan Pcr
%      secara manual (metode Ziegler-Nichols kalang tertutup),
%   3. menghitung Kcr dan Pcr dari model diskrit sebagai pembanding,
%   4. mensimulasikan parameter Ziegler-Nichols kalang tertutup dan kustom.

clc; clear; close all;

%% 1. Model sistem (waktu dalam ms)
```

```
K     = 0.85;      % penguatan model kecepatan (Modul 1)
Tau   = 60;        % konstanta waktu (ms)
Theta = 30;        % waktu tunda (ms)
C     = 0.02;      % konversi kecepatan ke laju posisi (%/ms per %), dari Percobaan 2
Ts    = 10;        % waktu cacah (ms), gunakan waktu cacah pertama

UMax     = 40;     % batas aksi kontrol (%)
Awal     = 50;     % posisi awal (%)
SetPoint = 70;     % set point posisi (%)
TSim     = 3000;   % durasi simulasi (ms)

%% 2. Kalang terbuka: masukan step 20%
[t, p] = simulasi(K, Tau, Theta, C, Ts, Awal, 0, TSim, 0, 0, 0, UMax, 20);
figure('Name', 'Kalang terbuka');
plot(t, p, 'LineWidth', 1.2); grid on;
xlabel('Waktu (ms)'); ylabel('Posisi (%, tanpa batas 0-100)');
title('Respons posisi kalang terbuka untuk masukan step 20%');

%% 3. Mencari Kcr dan Pcr secara manual
% Naikkan KpUji sampai respons berosilasi dengan amplitudo tetap.
KpUji = 1.0;
[t, p] = simulasi(K, Tau, Theta, C, Ts, Awal, SetPoint, TSim, KpUji, 0, 0, Inf, []);
figure('Name', 'Uji Kcr');
stairs(t, p); grid on; yline(SetPoint, 'k--');
xlabel('Waktu (ms)'); ylabel('Posisi (%)'); title(sprintf('Kontrol P, Kp = %g', KpUji));

%% 4. Pembanding: Kcr dan Pcr dari model diskrit
[Kcr, Pcr] = titik_kritis(K, Tau, Theta, C, Ts);
fprintf('Dari model diskrit: Kcr = %.3f, Pcr = %.1f ms\n', Kcr, Pcr);

%% 5. Ziegler-Nichols kalang tertutup
% baris: P, PI, PID, PID overshoot kecil, PID tanpa overshoot; kolom: Kp, Ti, Td
ZN = [ 0.5*Kcr,   Inf,      0;
       0.45*Kcr,  Pcr/1.2,  0;
       0.6*Kcr,   Pcr/2,    Pcr/8;
       0.33*Kcr,  Pcr/2,    Pcr/3;
       0.2*Kcr,   Pcr/2,    Pcr/3 ];
Nama = {'P', 'PI', 'PID', 'PID OS kecil', 'PID tanpa OS'};
figure('Name', 'Ziegler-Nichols kalang tertutup'); hold on; grid on;
fprintf('\n%-13s %8s %8s %7s %9s %8s | %7s %7s %6s %7s %7s\n', 'Kontrol', 'Kp', 'Ti', 'Td', 'Ki', 'Kd', ...
    'td', 'tr', 'OS', 'ts', 'offset');
for j = 1:5
    Kp = ZN(j, 1);
    if isinf(ZN(j, 2)), Ki = 0; else, Ki = Kp / ZN(j, 2); end
    Kd = Kp * ZN(j, 3);
    [t, p] = simulasi(K, Tau, Theta, C, Ts, Awal, SetPoint, TSim, Kp, Ki, Kd, UMax, []);
    P = kinerja(t, p, Awal, SetPoint);
    fprintf('%-13s %8.3f %8.1f %7.1f %9.5f %8.2f | %7.1f %7.1f %6.1f %7.1f %7.2f\n', ...
        Nama{j}, Kp, ZN(j,2), ZN(j,3), Ki, Kd, P);
    stairs(t, p, 'DisplayName', Nama{j});
end
yline(SetPoint, 'k--', 'HandleVisibility', 'off');
xlabel('Waktu (ms)'); ylabel('Posisi (%)'); legend('Location', 'southeast');

%% 6. Parameter kustom
KpKustom = 0.6; KiKustom = 0; KdKustom = 20;
```

```
[t, p] = simulasi(K, Tau, Theta, C, Ts, Awal, SetPoint, TSim, KpKustom, KiKustom,
    KdKustom, UMax, []);
P = kinerja(t, p, Awal, SetPoint);
fprintf('\nKustom: td=%.1f tr=%.1f OS=%.1f ts=%.1f offset=%.2f\n', P);
figure('Name', 'Kustom'); stairs(t, p); grid on; yline(SetPoint, 'k--');
xlabel('Waktu (ms)'); ylabel('Posisi (%)'); title('Parameter kustom');

%% Fungsi bantu
function [tk, pk] = simulasi(K, Tau, Theta, C, Ts, P0, SP, TSim, Kp, Ki, Kd, UMax,
    UBuka)
    % Bila UBuka tidak kosong, simulasi kalang terbuka dengan masukan UBuka.
    N = TSim; d = round(Theta);
    ubuf = zeros(N + d + 1, 1);
    v = 0; p = P0; u = 0; Acc = 0; LastE = 0; First = true;
    nk = floor((N - 1) / Ts) + 1; tk = zeros(nk, 1); pk = zeros(nk, 1); i = 0;
    for k = 0:N-1
        if mod(k, Ts) == 0
            if isempty(UBuka)
                e = SP - p;
                de = 0; if ~First, de = (e - LastE) / Ts; end
                First = false;
                u = Kp*e + Ki*(Acc + e*Ts) + Kd*de;
                if u > UMax
                    u = UMax; if e < 0, Acc = Acc + e*Ts; end
                elseif u < -UMax
                    u = -UMax; if e > 0, Acc = Acc + e*Ts; end
                else
                    Acc = Acc + e*Ts;
                end
                LastE = e;
            else
                u = UBuka;
            end
            i = i + 1; tk(i) = k; pk(i) = p;
        end
        ubuf(k + 1 + d) = u;
        v = v + (K*ubuf(k + 1) - v) / Tau;   % kecepatan (%)
        p = p + C*v;                         % posisi (%), langkah 1 ms
    end
end

function [Kcr, Pcr] = titik_kritis(K, Tau, Theta, C, Ts)
    % Gain P terkecil yang membuat model diskrit (dengan waktu tunda) tidak stabil
    a = exp(-Ts/Tau);
    A = [1, C*Tau*(1 - a); 0, a];
    B = [C*K*(Ts - Tau*(1 - a)); K*(1 - a)];
    d = round(Theta/Ts);
    Kcr = NaN; Pcr = NaN;
    for Kp = 0.005:0.005:50
        M = matriks_tertutup(A, B, d, Kp);
        ev = eig(M);
        [rho, idx] = max(abs(ev));
        if rho > 1
            Kcr = Kp;
            Pcr = 2*pi / abs(angle(ev(idx))) * Ts;
            return;
        end
    end
end

function M = matriks_tertutup(A, B, d, Kp)
```

```
132     % Keadaan: [posisi; kecepatan; u[k-1]; ...; u[k-d]], u[k] = -Kp*posisi
133     n = 2 + d;
134     M = zeros(n);
135     M(1:2, 1:2) = A;
136     if d == 0
137         M(1:2, 1) = M(1:2, 1) - Kp*B;
138     else
139         M(1:2, n) = B;
140         M(3, 1) = -Kp;
141         for i = 2:d
142             M(2 + i, 1 + i) = 1;
143         end
144     end
145 end
146 
147 function P = kinerja(t, y, Awal, SP)
148     % [delay time, rise time 10-90%, overshoot, settling time 2%, offset] terhadap
        perubahan set point
149     s = (y - Awal) / (SP - Awal);
150     sf = mean(s(end - max(1, round(0.1*numel(s))) + 1:end));
151     td = waktu_pertama(t, s >= 0.5);
152     tr = waktu_pertama(t, s >= 0.9) - waktu_pertama(t, s >= 0.1);
153     OS = max(0, (max(s) - 1) * 100);
154     luar = find(abs(s - 1) > 0.02);
155     if isempty(luar), ts = t(1); elseif luar(end) < numel(t), ts = t(luar(end) + 1);
         else, ts = NaN; end
156     P = [td, tr, OS, ts, (1 - sf)*(SP - Awal)];
157 end
158 
159 function tw = waktu_pertama(t, Kondisi)
160     k = find(Kondisi, 1);
161     if isempty(k), tw = NaN; else, tw = t(k); end
162 end
```